

Relatório da Atividade 1 – Processamento Básico de Imagens

Aluno: Yago Melo Da Costa

Tema do trabalho final: Moda

Data: 08/04/2026

1. Objetivo

Implementou manualmente seis técnicas fundamentais de processamento digital de imagens (PDI), utilizando a biblioteca **PIL (Pillow)** para carregamento e salvamento, e **NumPy** para manipulação de arrays. Todas as operações (convolução, correção gama, média ponderada, transformações de intensidade, mosaico e quantização) foram desenvolvidas sem o uso de funções prontas de processamento, focando na manipulação direta da matriz de pixels.

2. Imagens Utilizadas

As imagens foram selecionadas com base no tema **moda**, buscando representar diferentes texturas e contrastes:

- **imagem1.jpeg**: Modelo masculino em traje casual, fundo claro neutro.



- **imagem2.jpeg:** Peça de vestuário (vestido) com detalhes texturizados.



- **imagem3.jpeg:** Conjunto de vestuário unisex completo.



- **imagem4.jpeg:** Passarela de moda verão com iluminação contrastante.



3. Desenvolvimento e Resultados

Questão 1 – Efeito de Esboço a Lápis

Implementação: A conversão para escala de cinza foi feita via fórmula de luminância ($Y = 0,299R + 0,587G + 0,114B$). Foi aplicado um kernel Gaussiano 21×21 ($\sigma = 5$) através de convolução manual. O efeito final foi obtido pela divisão pixel a pixel da imagem original em tons de cinza pela versão suavizada.

```
import numpy as np
```

```
from PIL import Image
```

```
def rgb_to_grayscale(img_array):
```

```
    r, g, b = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
```

```
    return (0.299 * r + 0.587 * g + 0.114 * b).astype(np.uint8)
```

```
def gaussian_kernel(size, sigma=1):
```

```
    kernel = np.fromfunction(
```

```

        lambda x, y: (1/(2*np.pi*sigma**2)) * np.exp(-((x-(size-1)/2)**2 + (y-(size-1)/2)**2) /
(2*sigma**2)),
        (size, size)
    )
    return kernel / np.sum(kernel)

```

```

def convolve(img, kernel):
    k_size = kernel.shape[0]
    pad = k_size // 2
    img_pad = np.pad(img, pad, mode='edge')
    output = np.zeros_like(img)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            region = img_pad[i:i+k_size, j:j+k_size]
            output[i,j] = np.sum(region * kernel)
    return output

```

```

def pencil_sketch(image_path, output_path, kernel_size=21, sigma=5):
    img = Image.open(image_path).convert('RGB')
    img_array = np.array(img)
    gray = rgb_to_grayscale(img_array)
    kernel = gaussian_kernel(kernel_size, sigma)
    blurred = convolve(gray, kernel)
    epsilon = 1e-8
    sketch = (gray.astype(np.float32) / (blurred.astype(np.float32) + epsilon)) * 255
    sketch = np.clip(sketch, 0, 255).astype(np.uint8)
    Image.fromarray(sketch).save(output_path)
    return gray, blurred, sketch

```

Análise: O resultado (arquivo questao1_sketch.jpg) destaca os contornos da silhueta e as dobras do tecido, simulando com precisão um traçado artístico. O uso do kernel Gaussiano largo foi essencial para suprimir detalhes irrelevantes e focar nas bordas principais.



Questão 2 – Correção Gama

Implementação: Os pixels foram normalizados para o intervalo $[0,1]$ e submetidos à potência inversa do valor de gama ($B = A^{(1/\gamma)}$).

```
import numpy as np
```

```
from PIL import Image
```

```
def gamma_correction(image_path, output_path, gamma):
```

```
    img = Image.open(image_path).convert('L') # 'L' = escala de cinza
```

```
    img_array = np.array(img, dtype=np.float32)
```

```
normalized = img_array / 255.0
corrected = np.power(normalized, 1.0/gamma) * 255.0
corrected = np.clip(corrected, 0, 255).astype(np.uint8)
Image.fromarray(corrected).save(output_path)
return corrected
```

Resultados Observados:

Gama (γ)	Efeito Observado
0,5	Aumento expressivo do brilho; detalhes em tecidos pretos tornam-se visíveis.
1,0	Identidade (sem alteração).
1,5	Leve escurecimento; aumento do contraste perceptual.
2,5	Escurecimento acentuado (subexposição); útil para realçar pontos de luz intensos.







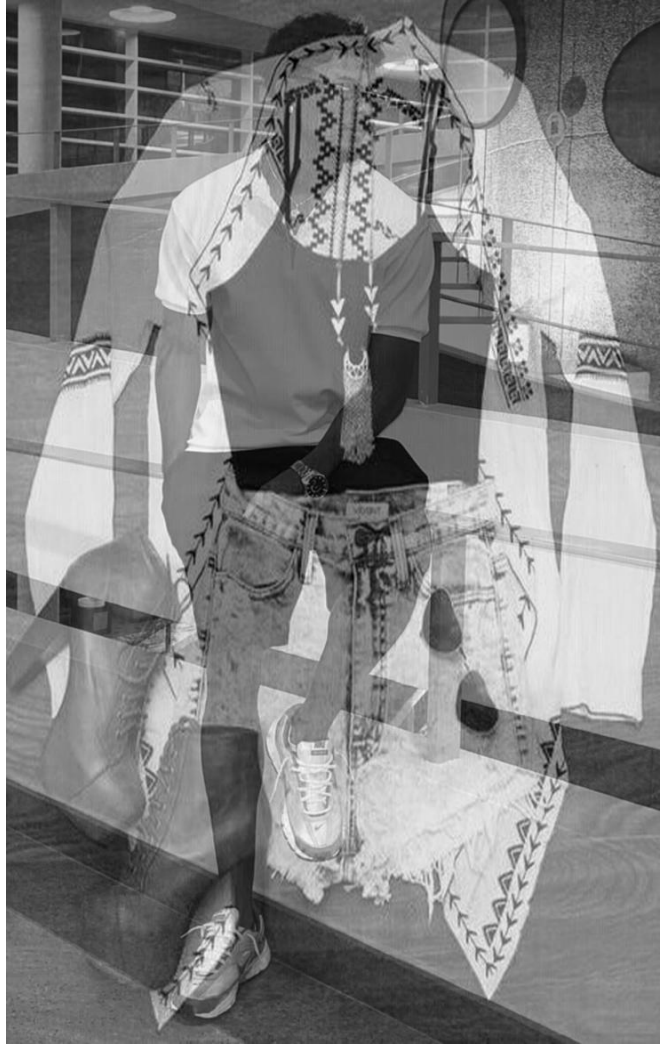
Questão 3 – Média Ponderada de Duas Imagens

Implementação: As imagens foram redimensionadas para compatibilidade e combinadas pela fórmula: $\text{Resultado} = \alpha \cdot \text{img1} + (1 - \alpha) \cdot \text{img2}$.

```
import numpy as np
from PIL import Image

def weighted_blend(img1_path, img2_path, output_path, alpha=0.5):
    img1 = Image.open(img1_path).convert('L')
    img2 = Image.open(img2_path).convert('L')
    if img1.size != img2.size:
        img2 = img2.resize(img1.size, Image.Resampling.LANCZOS)
        print(f"Aviso: imagem2 redimensionada de {img2.size} para {img1.size}")
    arr1 = np.array(img1, dtype=np.float32)
    arr2 = np.array(img2, dtype=np.float32)
    blended = (alpha * arr1 + (1 - alpha) * arr2).astype(np.uint8)
    Image.fromarray(blended).save(output_path)
    return blended
```

Análise: Com $\alpha = 0,5$, obteve-se uma fusão perfeita entre a modelo e a textura do tecido da segunda imagem. Com $\alpha = 0,7$, a modelo ganha destaque, mantendo a textura como um "overlay" sutil, técnica comum em editoriais de moda.





Questão 4 – Transformações Sequenciais

Implementação: A imagem passou por cinco etapas: Negativo, Reescala (intervalo 100-200), Inversão de linhas pares, Espelhamento da metade superior e Espelhamento vertical total.

```
import numpy as np
```

```
from PIL import Image
```

```
def negative(img_array):
```

```
    return 255 - img_array
```

```
def rescale_intensity(img_array, new_min=100, new_max=200):
```

```
old_min, old_max = 0, 255
img_float = img_array.astype(np.float32)
rescaled = ((img_float - old_min) / (old_max - old_min)) * (new_max - new_min) + new_min
return rescaled
```

```
def reverse_even_rows(img_array):
    output = img_array.copy()
    for i in range(0, output.shape[0], 2):
        output[i] = output[i, ::-1]
    return output
```

```
def mirror_top_to_bottom(img_array):
    h = img_array.shape[0]
    half = h // 2
    output = img_array.copy()
    output[half:] = output[:half][::-1]
    return output
```

```
def vertical_flip(img_array):
    return img_array[::-1, :]
```

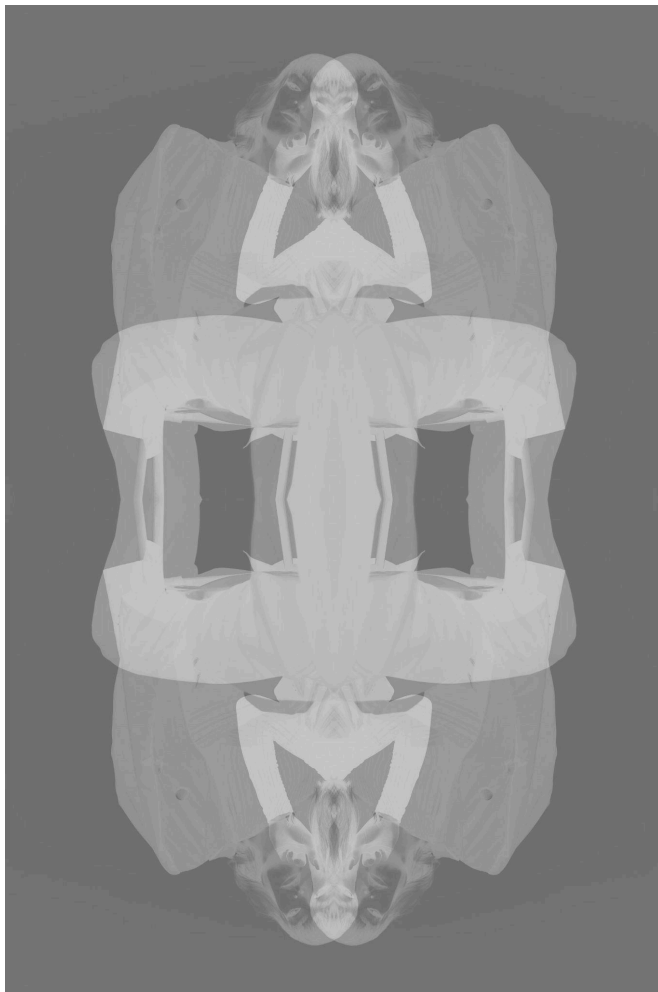
```
def apply_all_transformations(image_path, output_path):
    img = Image.open(image_path).convert('L')
    arr = np.array(img, dtype=np.float32)
    arr = negative(arr)
    arr = rescale_intensity(arr)
    arr = reverse_even_rows(arr)
    arr = mirror_top_to_bottom(arr)
    arr = vertical_flip(arr)
```

```
arr = np.clip(arr, 0, 255).astype(np.uint8)

Image.fromarray(arr).save(output_path)

return arr
```

Análise: O resultado final (questao4_transformed.jpg) é uma imagem abstrata e simétrica. A reescala de intensidade para [100, 200] resultou em uma imagem de baixo contraste, conferindo uma estética de "filme antigo" ou lavagem ácida, enquanto as inversões de linha criaram uma textura digital ruidosa.



Questão 5 – Mosaico 4×4

Implementação: A imagem foi fatiada em 16 blocos. A reordenação seguiu o padrão de transposição de colunas, movendo os blocos para novas coordenadas conforme o roteiro da atividade.

```
import numpy as np
from PIL import Image

def create_mosaic(image_path, output_path, block_size):
    img = Image.open(image_path).convert('L')
    img_array = np.array(img)
    h, w = img_array.shape
    block_h, block_w = h // block_size, w // block_size
    blocks = []
    for i in range(block_size):
        for j in range(block_size):
            block = img_array[i*block_h:(i+1)*block_h, j*block_w:(j+1)*block_w]
            blocks.append(block)
    new_order = [0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
    mosaic = np.zeros_like(img_array)
    for idx, new_idx in enumerate(new_order):
        i, j = divmod(idx, block_size)
        mosaic[i*block_h:(i+1)*block_h, j*block_w:(j+1)*block_w] = blocks[new_idx]
    Image.fromarray(mosaic).save(output_path)
    return mosaic
```

Análise: A técnica desconstrói a peça de moda original, transformando a vestimenta em um padrão geométrico. É uma prova de conceito eficaz sobre a manipulação de índices em matrizes multidimensionais.



Questão 6 – Quantização de Níveis de Cinza

Implementação: A redução de tons foi feita calculando o "passo" ($256/\text{n\acute{v}eis}$) e arredondando os valores dos pixels para o nível inferior mais próximo.

```
import numpy as np
```

```
from PIL import Image
```

```
def quantize(image_path, output_path, levels):
```

```
    img = Image.open(image_path).convert('L')
```

```
    img_array = np.array(img, dtype=np.float32)
```

```
    step = 256 / levels
```

```
    quantized = np.floor(img_array / step) * step
```

```
    quantized = np.clip(quantized, 0, 255).astype(np.uint8)
```



```
Image.fromarray(quantized).save(output_path)  
  
return quantized
```

Análise: A transição de 256 para 2 níveis revela como a informação visual é perdida. Em 8 níveis, a imagem adquire um estilo "posterizado". Em 2 níveis (binarização), restou apenas a silhueta da peça de moda, o que é útil para a criação de máscaras de segmentação.

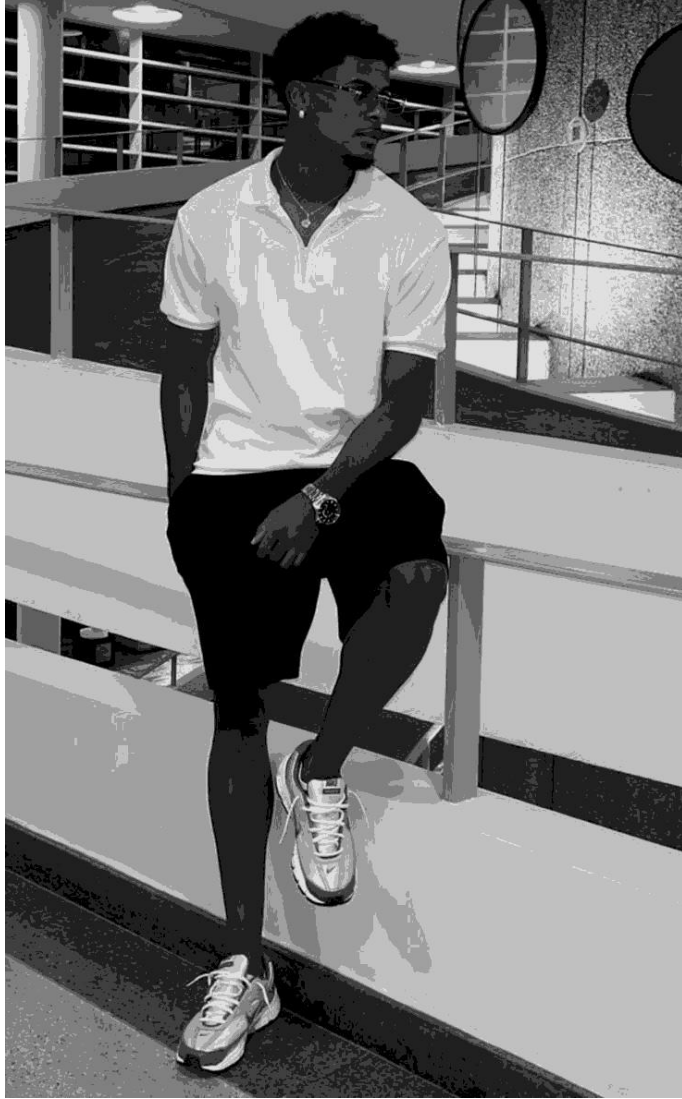














Código Main.py

```
from pathlib import Path
from Questao1 import pencil_sketch
from Questao2 import gamma_correction
from Questao3 import weighted_blend
from Questao4 import apply_all_transformations
from Questao5 import create_mosaic
from Questao6 import quantize

def main():
    base_path = Path("images")
    img1 = base_path / "imagem1.jpeg"
    img2 = base_path / "imagem2.jpeg"
```



```
img3 = base_path / "imagem3.jpeg"
img4 = base_path / "imagem4.jpeg"

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

print("Executando Questão 1: Esboço a lápis...")
sketch_output = output_dir / "questao1_sketch.jpg"
pencil_sketch(str(img1), str(sketch_output), kernel_size=21, sigma=5)
print(f" -> Salvo em {sketch_output}")

print("Executando Questão 2: Correção gama...")
gammas = [0.5, 1.0, 1.5, 2.0, 2.5]
for gamma in gammas:
    gamma_output = output_dir / f"questao2_gamma_{gamma}.jpg"
    gamma_correction(str(img2), str(gamma_output), gamma)
    print(f" -> Gamma {gamma} salvo em {gamma_output}")

print("Executando Questão 3: Média ponderada...")
alphas = [0.3, 0.5, 0.7]
for alpha in alphas:
    blend_output = output_dir / f"questao3_blend_alpha_{alpha}.jpg"
    weighted_blend(str(img1), str(img2), str(blend_output), alpha)
    print(f" -> Alpha {alpha} salvo em {blend_output}")

print("Executando Questão 4: Transformações sequenciais...")
trans_output = output_dir / "questao4_transformed.jpg"
apply_all_transformations(str(img3), str(trans_output))
print(f" -> Salvo em {trans_output}")

print("Executando Questão 5: Mosaico 4x4...")
mosaic_output = output_dir / "questao5_mosaic.jpg"
create_mosaic(str(img4), str(mosaic_output), block_size=4)
print(f" -> Salvo em {mosaic_output}")

print("Executando Questão 6: Quantização...")
niveis = [256, 64, 32, 16, 8, 4, 2]
for levels in niveis:
    quant_output = output_dir / f"questao6_quant_{levels}levels.jpg"
    quantize(str(img1), str(quant_output), levels)
    print(f" -> {levels} níveis salvo em {quant_output}")

print("\nTodas as questões foram processadas com sucesso!")
```

```
if __name__ == "__main__":  
    main()
```

4. Conclusão

O trabalho demonstrou que é possível realizar manipulações complexas de imagem utilizando apenas lógica matemática e acesso direto aos arrays de pixels. As técnicas aplicadas ao tema de moda mostraram-se úteis tanto para fins estéticos (filtros e efeitos) quanto para análise técnica (realce de texturas e binarização de moldes).

5. Referências

- GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4th ed. Pearson, 2018.
- Documentação Pillow (PIL): <https://pillow.readthedocs.io/>